

软件设计下

策略模式

- 优势
 - 这个模式让类跟算法独立开来。
 - 类在运行的时候把算法委托给策略对象去处理。
 - 可以说算法的选择不是在编译的时候定死的。代码库维护起来更容易。它也很容易扩展
- 主要挑战
 - 增加上下文类会导致应用程序里的对象变多。
 - 应用程序的用户必须知道有哪些不同的策略，不然输出结果可能会吓他们一跳。
 - 所以客户端代码和不同策略的实现之间存在紧密的联系。当引入一个新的行为或者算法时，可能也需要去修改客户端的代码

模板方法模式

- 优势
 - 你可以控制算法的流程。客户端无法更改它们。
 - 注意抽象类 BasicEngineering 中的 completeCourse 被标记了 final 关键字。通用的操作被放置在一个中心位置，例如在抽象类中。子类只能重新定义变化的部分，这样你就可以避免重复的代码。
- 主要挑战
 - 客户端代码无法指导步骤的顺序。
- 与建造者模式区别
 - 建造者是一种创建型设计模式。在建造者模式中，客户或顾客是老板，他们可以控制算法的顺序。
 - 模板方法是一种行为设计模式
 - 在模板方法模式中，你是老板，你把代码放在一个中心位置，并且只提供相应的行为。
 - 对执行流程有绝对控制权

迭代器模式

- 优点
 - 可以在 “不了解对象内部结构细节” 的情况下遍历它。也就是说，

```
public class IteratorPatternExample {
    public static void main(String[] args) {

        // 场景 1: 处理 Arts (数组)
        Subjects arts = new Arts();
        Iterator it1 = arts.createIterator();
        printAll(it1); // 调用通用方法

        // 场景 2: 处理 Science (ArrayList)
        Subjects science = new Science();
        Iterator it2 = science.createIterator();
        printAll(it2); // 调用同一个通用方法!
    }

    // 【核心点】: 这个方法完全不知道内部是数组还是列表
    // 它只认识 "Iterator" 接口
    public static void printAll(Iterator it) {
        while (it.hasNext()) {
            System.out.println(it.next());
        }
    }
}
```
 - 可以用不同的方式去遍历同一个集合。你甚至可以提供一种实现，让它同时支持多种不同的遍历方式
- 主要挑战
 - 在遍历或者迭代的过程中，不应该对核心的架构进行任何意外的修改
 - (假设现在你有一个新需求: “我想倒序遍历”，从最后一个元素读到第一个)。
 - (就是当你正在用 'iterator' 挨个读数据 (遍历) 的时候，你突然对这个数组进行了“增加”或者“删除”操作)。

桥接模式

- 主要优势
 - 实现不绑定到抽象上。
 - 抽象和实现都可以独立拓展。
 - 具体类独立于接口实现者类 (即，其中一个的变化不会影响另一个)。你还可以以不同的方式改变接口和具体实现。
- 挑战
 - 整体结构可能会变得复杂。
 - 有时它会与适配器模式混淆。(适配器模式的关键目的是仅处理不兼容的接口。)

访问者模式

- 优势
 - 需要给一组对象添加新的操作，但又不想改变它们对应的类时。
 - 如果这些操作经常发生变化，需要更改各种操作的逻辑
 - 如果不在意封装性能。

命令模式

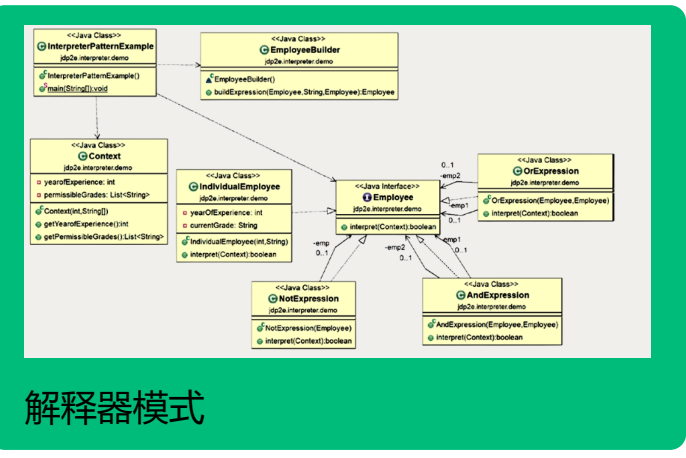
- 优点
 - 请求的创建和最终的执行是解耦的，客户端不需要知道 Invoker 具体是如何执行操作的
 - 可以创建一系列的命令组合
 - 支持撤销和重做操作。
- 挑战
 - 为了支持更多的命令，你需要创建更多的类，随着时间推移，维护起来会比较困难
 - 当发生错误情况时，如何处理错误或者决定如何处理返回值会变得比较棘手
 - 客户端可能想知道这些情况，但因为你把命令和客户端代码解耦了，这些情况就很难处理。
 - 特别是在多线程环境下，如果 Invoker 运行在不同的线程中，这个挑战会变得更加明显。

状态模式

- 中介者模式
 - 优点
 - 降低系统中对象通信的复杂性
 - 促进了松耦合。它减少了系统中的子类数量
 - 以把多对多的关系替换成一对多的关系，所以读起来和理解起来更容易
 - 缺点
 - 实现恰当的封装比较棘手。如果你把太多的逻辑放进去，中介者对象的架构可能会变得很复杂
 - 外观模式和中介者模式
 - 中介者模式描述为多路复用的外观模式。
 - 在中介者模式中，你不是在处理单个对象的接口，而是在多个对象之间建立一个多路复用的接口，以提供平滑的过渡。
- 状态模式和策略模式
 - 适用场景
 - 如果你想设计那种对象的行为会随着内部状态改变而改变的功能，这个模式就很有用
 - 考虑顾客买在线票然后在稍后阶段退票的场景。退款金额可能会根据不同的条件而变化
 - 比如你**在距离出发多少天之前取消车票
 - 例子
 - 策略模式提供了一个比生成子类更好的替代方案，就是改变内部某个实现算法
 - 不同类型的行为被封装在一个状态对象里，上下文会把工作委托给这些状态中的任何一个
 - 当上下文的内部状态发生变化时，它的行为也会随之改变
 - ```
class TV {
 // true when before
 public void powerOn() {
 // powerOn() method
 }
 // true when after
 public void powerOff() {
 // powerOff() method
 }
 // true when on
 public void powerOn() {
 // powerOn() method
 }
 // true when off
 public void powerOff() {
 // powerOff() method
 }
}
```
    - 状态模式还能帮我们在某些情境下避免写大量的 if 条件语句
  - 共同点
    - 两者都能促进组合和委托的使用
  - 优点
    - 遵循开闭原则，你可以很容易地添加新的状态和新的行为
    - 扩展状态行为也很容易
    - 它减少了 if-else 语句的使用，也就是说降低了条件逻辑的复杂度 (参考问题 3 的答案)。
  - 缺点
    - 状态模式也被称为状态对象。所以，你可以认为更多的状态需要更多的代码，显而易见的副作用就是你的维护难度会增加。

备忘录模式

- 主要优势
  - 总是可以丢弃不需要的更改，并将它恢复到预期的或稳定的状态
  - 不需要牺牲参与此模型的关键对象的封装性
  - 保持了高内聚
  - 提供了一种简单的恢复技术
- 主要挑战
  - 大量的备忘录需要更多的存储空间
  - 它们给 Caretaker 增加了额外的负担，同时也增加了维护成本
  - 保存状态的额外时间会降低系统的整体性能
- 与命令模式的讨论
  - 什么时候用命令模式
    - 比如说做完一个操作，想直接反向撤回 (也就是 50 + 10 = 60，所以要回去，你就做 60 - 10 = 50)
    - 这种类型的操作中，我们不需要存储以前的状态
  - 备忘录模式
    - 在命令模式中，不一定要存储命令。一旦你执行了一个命令，它的工作就完成了。如果你不支持“撤销”操作，你可能根本没兴趣存储这些命令。
    - 需要存储操作之前对象的状态。在这种情况下，备忘录就是你的救星
    - 在备忘录模式中存储备忘录对象是强制性的，这样你才能回滚到以前的状态；
    - 一个应用程序可以使用“这两种模式来支持撤销操作”



- 定义
  - 给定一种语言，定义其文法的一种表示，并定义一个解释器，该解释器使用该表示来解释语言中的句子。它通过将文法规则映射为类来解析和处理特定语言
- 优势
  - 能够深度参与到定义语言语法的过程中
  - 也清楚该怎么表示和解释这些语句
  - 还能随时修改和扩展你的语法规则
- 困难
  - 怎么解释这些表达式，你拥有完全的控制权和自由
  - 工作量太大了。还有就是维护复杂的语法会很麻烦
  - 需要为每一条不同的规则都创建和维护单独的类